

Table of Contents

Background	2
Functional Requirements	2
Feature 1: Destination search	2
Feature 2: Hotel Search Results	3
Feature 3: Hotel Room Details	3
Feature 4: Booking data	4
Non-functional requirements	5
Testing	5
Speed	5
Data security	5
Payment information	5
Personal information	6
Appendix 1 - Data	7
Appendix 2 - Comparing hotel prices data	Error! Bookmark not defined.
Appendix 3 - Working with Ascenda's data	7
1. Setup	8
2. Static Data	9
2.1. Destinations	9
2.2. Hotels	Error! Bookmark not defined.
3. Using the Endpoints	11
3.1. /api/hotels/prices	11
3.2. /api/hotels/:id/prices	Error! Bookmark not defined.
3.3. /api/hotels	12
3.4. /api/hotels/:id	Error! Bookmark not defined.

Telegram link

<https://t.me/+Oq7UwgLwISxhMjE1>

FAQs

<https://docs.google.com/document/d/1AKnKNMT9qrgrA17XsK6nzlggFFF20IFNG-z2T6euoBI/edit?tab=t.0>

Background

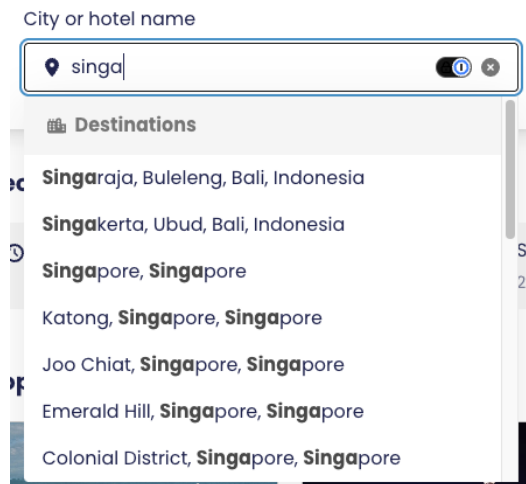
Ascenda powers whitelabelled hotel booking platforms on behalf of banks, airlines and loyalty programs. This means that customers can earn & redeem hotel night stays.

Functional Requirements

Feature 1: Destination search

You should provision a simple destination search feature. This is a text-based autocomplete search that should be fast from the customer's perspective. You're free to use any tools/ techniques that can facilitate a fast return of results.

Using the destination selected will allow you to search for hotels relevant to that destination.



Deliverable:

- A simple form with a destination search dropdown that can suggest you relevant places / destinations as you type a destination
- Dropdown results should be displayed as fast as possible (the faster the better)
- Incorporate a datepicker to allow users to select their dates of stay
- Incorporate a dropdown (or any other UI) for users to specify the number of guests & number of rooms (the format of API request params for guests & rooms are in Appendix 3.1)
- Must be able to process simple typos (e.g. Singapore → Singapore), you could choose to use an external library for this
- Form submission & validation, redirecting to Feature 2.

Feature 2: Hotel Search Results

Customers should be able to view hotel search results relatively quickly.

For a given destination, dates of stay and number of rooms/guests, a list of matching hotels and the cheapest room for each hotel would be given by our mock API. This is typically used to render a listings view of all available hotels and the cheapest room of each hotel.

Deliverables:

- A web page that displays a list of all available hotels for the selected parameters (e.g. destination, dates, no. of guests, no. of rooms) in Feature 1.
- Since the list of hotels would be very long, your web page might be laggy or freezing while rendering the list. Consider how you can solve this problem (hint: lazy loading? paginations?)
- A filter panel that could filter hotels displayed by star ratings, guest ratings, and price range.
- Sort the list of hotels based on price & ratings
- A select button for each list item where you can select a hotel and redirect to Feature 3.
- [Bonus] Build a map view to browse & select hotels from the map view

Feature 3: Hotel Room Details

For a given hotel (chosen from the list from feature 2), dates of stay, and number of rooms/guests, this API call will provide a list of all matching room types that can be provided by the specified hotel. This is typically used to render a view of all room listings that are available for booking in the specified hotel, as well as some additional information of the hotel.

Given that room rates between views could vary, consider how you would maintain the customer experience by ensuring the most accurate rates across views.

Deliverables:

- A web page that displays more detailed hotel information:
 - Hotel name, address, description, ratings, amenities, photos
- Shows a list of all available rooms for the selected hotel in Feature 2:
 - For each room, show the room's photos, type, description, amenities, prices
- A map that shows the hotel's location (notice that the mock API will return the longitude and latitude of the hotel in Appendix 3.3.4)
- Select button for each room option to redirect to Feature 4

Feature 4: Booking data

A customer will be making their booking and providing payment via a payment gateway. You are not required to create a mock payment gateway, but using a payment gateway would be a plus (+ it might help you handle some payment security). You can assume that a booking is successful once the user submits a valid form with the relevant guest information (see deliverables).

The data here is used to display booking information about the stay & for tracking customer support purposes. Here's a sample booking information:

Information	Description
Destination ID	-
Hotel ID	-
Booking display information	Number of nights, start date, end date, adults, children, message to hotel, room types
Price	The price paid for this booking
Booking reference	Your system's booking ID
Guest information	Salutation, first name, last name
Payee information	Payment ID, Payee ID

Deliverables:

- A form that collects the user's guest information:
 - First Name & Last Name
 - Phone Number & Email Address
 - Special requests to hotel
 - Payment information (credit card number, expiry date, CVV/CVC)
 - Billing address
- A submit button that calls a Create Booking API (you should implement the API yourself)
 - Integrate with Stripe API
 - Store booking details in database
- Booking information that you store in your database should look something like the table above. Make sure you have some privacy security in place (see non-functional requirements).
- After submitting the booking form page, you should show a confirmation page stating the booking has been confirmed with all the details previously collected. It would be good to show payment details like last 4 digits of the card used for payment.

Non-functional requirements

In this project, there're a couple of challenges you'd have to consider as we build a system to service our partners:

Testing

To have a robust frontend and backend system in any web product, the unit tests are important to ensure continuous development and integration is smooth. Regardless of the framework you choose for your backend and frontend system, you should have comprehensive unit tests to ensure the correct functionality of the entire application.

For the frontend part, it will be a bonus if you can implement certain end to end tests to ensure the web application is interactive and working as expected from the user's perspective.

Speed

As an ecommerce website, speed is of the essence here. Our competitors typically produce results in no more than a couple of seconds.

This speed is especially crucial for the destination/ hotels autocomplete. The industry standard is a near-instantaneous response, consider how this could be done.

Given the aggregated nature of supplier rates, it is natural for the hotel results to take longer to complete, still the first results should take no more than a few seconds

Full results can take up to 30 seconds depending on how long rates take to finalize.

Data security

Payment information

Any payment information collected should only be transient for the purposes of conducting the payment with the payment processor. You can store credit card numbers for display if you mask everything but the first 6 & last 4 digits.

You should not persist any private customer payment information except for the non-PII payment references with the processor.

1 common customer incident is a mistaken entry of data, or the triggering of a 3DS payment flow (think OTP) which reflects the need to at least store some payment information transiently until the user back in our site to complete or correct the transaction flow. Consider how you'd deal with the data securely. You can read more on credit card PII guidelines [here](#).

Personal information

GDPR (General Data Protection Regulation) allows for an end user to request that their information be destroyed. Similarly, partners can request for PII information to be destroyed/ made inaccessible.

You should have a plan in place for effective removal of user PII (name, email, phone number, birth date, etc.) as needed. A straightforward strategy is to implement a “Delete Account” feature which allows users to delete their information completely from your database.

Appendix 1 - Destination Data

[destinations.json](#) shared as a downloadable file.

You'll use this to help implement a basic but performant destination text search; The best travel sites can do this in under 100ms (just about the normal human reaction time when searching with autocomplete).

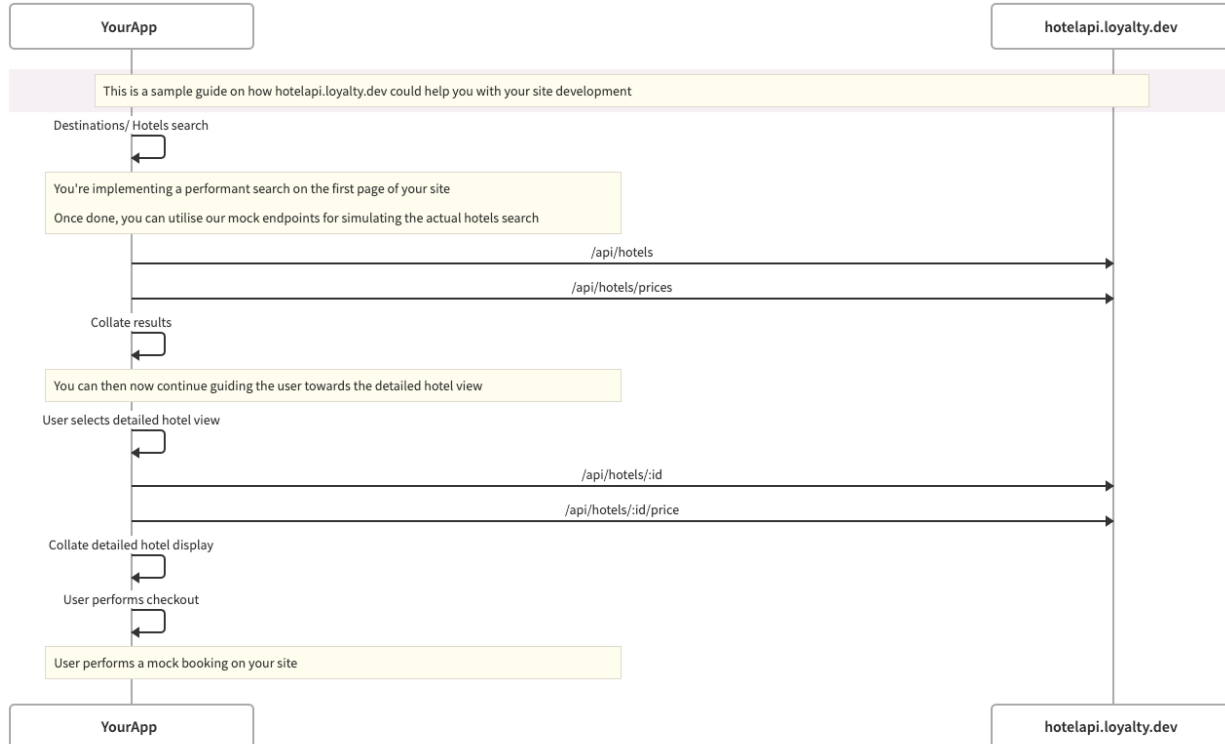
The destination "uid" is used in the next call as input to search Ascenda's static `/api/hotels` to fetch the full list of applicable hotels.

Appendix 2 - Working with Ascenda's data

We're exposing the APIs of our mock environment for you to work with for simplifying your travel site's functionalities.

There're a few things you'll have here to aid your implementation:

1. Static data for destinations & hotels
2. API guide for using our pricing endpoints



1. Overview

Reference site: refer to briefing recording

Your challenge here to to help us improve it by demonstrating your implementation of a performant destinations & hotels search.

Our static data discussed below will provide the baseline for you to integrate against our mock pricing APIs

Mock Pricing APIs endpoints

1. hotelapi.loyalty.dev/api/hotels/prices: Pricing search for a given destination
2. hotelapi.loyalty.dev/api/hotels/:id/price: Price search for a given hotel

These API endpoints provision a polling based price search capability for you to integrate to your destination & hotels search.

The static data provisioned to your contains the destination and hotel IDs necessary for interfacing with this search.

Mock Static Data endpoints

Your search results should contain details about the hotels on top of pricing. To simplify your efforts further, we've provisioned endpoints for you to get static hotel information too.

1. hotelapi.loyalty.dev/api/hotels: Static information of hotels belonging to a destination
2. hotelapi.loyalty.dev/api/hotels/:id: Static information for a given hotel

You'll use this to compile the results to be displayed for the end user

2. Data Structure

The static data details here provide you with a baseline for designing your search.

2.1. Destinations

The user should be able to search for destinations/ hotels using parts of the name.

Field	Description
UID	Destination ID; Your search will need to use this ID for pulling results
Term	Name of the destination
Lat	Latitude
Lng	Longitude
State	State of the country (if applicable)
Type	City or Hotel

2.2. Hotels

Field	Description
UID	Hotel ID; Your search will need to use this ID for pulling results
Name	
Address	
Rating	Hotel star rating
LonLat	
Phone Number	
Contact Email	
Fax Number	
Amenities	
Description	
Postal Code	
City	
State	
Country Code	
Image Count	
Primary destination ID	If present, this tells you the primary destination this hotel is classed under

3. Using the Endpoints

Our endpoints are meant to help you simplify the app implementation for you. Here're details about the API for you to integrate to your initial destinations & hotels search to the results of the search.

3.1. /api/hotels/prices

This endpoint returns you the hotel prices for a given destination

E.g.

https://hotelapi.loyalty.dev/api/hotels/prices?destination_id=RsBU&checkin=2026-10-01&checkout=2026-10-07&lang=en_US¤cy=SGD&country_code=SG&guests=2&partner_id=1089&landing_page=wl-acme-earn&product_type=earn

Request query params

Field	Description
destination_id	Destination ID for the location searched for
checkin	YYYY-MM-DD format Has to be a minimum of 3 days in advance
checkout	YYYY-MM-DD format Has to be later than checkin date
lang	E.g. "en_US"
currency	ISO code for currencies
guests	Number of guests staying per room If there are more than one room, append with ` ` separator E.g. 1 room 2 guests → guests=2 2 rooms 2 guests → guests=2 2 4 rooms 3 guests → guests=3 3 3 3
partner_id	This is unique to Kaligo for generating the loyalty points earned for a given program. This is not relevant to your project here so stub in a value for now. Populate a value of `1089` to stub this value else you'll not get any results
landing_page	Hardcode to `wl-acme-earn` for this case study

product_type	Hardcode to `earn` for this case study
--------------	--

Response attributes

Pricing responses here allow you to match the prices for available hotels to your source of hotels for a given destination

The IDs and values here are also used for navigating to the detailed hotels page of the site.

Field	Description
id	Hotel ID
searchRank	Internal ranking of hotel pricing results (by which is the best value price wise + loyalty points earned)
price	Price of hotel room (incl. of taxes and fees)
market_rates	Prices of equivalent rooms for a given supplier

3.2. /api/hotels

This endpoint supports you with info about the hotels belonging to a particular destination for the end user.

E.g. https://hotelapi.loyalty.dev/api/hotels?destination_id=RsBU

Request query param

Field	Description
destination_id	Once you've got a destination ID, you can use this to retrieve hotels for a given destination

Response attributes

Field	Description
id	Hotel ID
name	
latitude	For maps display purposes
longitude	For maps display purposes
address	
rating	Hotel star rating

categories	Hotel categories type
description	
amenities	
image_details	The suffix, count & prefix is used to generate the image links for end users

3.3. /api/hotels/:id/prices

This endpoint returns you the price for a given hotel. This is useful for displays involving a detailed hotels page.

E.g. https://hotelapi.loyalty.dev/api/hotels/QDaO/price?destination_id=RsBU&checkin=2026-10-10&checkout=2026-10-17&lang=en_US¤cy=SGD&country_code=SG&guests=2&partner_id=1089&landing_page=wl-acme-earn&product_type=earn

Request query params

This is the same as /api/hotels/prices as the same search params are used to determine prices

Response attributes

You receive the results containing prices of various room types under the top-level “rooms” attribute.

Key	Booking key used to uniquely identify the booking environment for the customer should this room be checked out for payment later
Room Normalized Description	Room type
Free Cancellation	Whether you get free refunds up till the stay itself (or up till a certain date)
Description	
Long Description	Exact details of a given room
Images	
Amenities	
Price	
Market Rates	Comparison of other suppliers with nearby searches.

3.4. /api/hotels/:id

This endpoint returns the static details for a given hotel

E.g. <https://hotelapi.loyalty.dev/api/hotels/QDaO>

Request params

This is as described in the endpoint definition above; Just the hotel ID is enough.

Response attributes

This is the same as that of 3.2